

KOBLY · INTERNAL HANDOFF DOCUMENT

The /wizard lead form

A complete functional and visual specification of the 5-step moving-quote wizard, written for a rebuild in Django + server-rendered HTML/SCSS/vanilla JS (no React/Next.js).

Source analyzed: `app/wizard/page.tsx` , `components/wizard/MapPickerOverlay.tsx` ,
`components/marketing/FullscreenOverlay.tsx` , `lib/cities.ts` , `app/globals.css` , `app/layout.tsx`
Prepared: 2026-08-20

o. Contents & priority

1. What this page does (product view)
2. Critical fact: there is currently no backend
3. Page shell & layout (desktop vs. mobile)
4. State model — the full data shape
5. Step 1 — Address + map (the highest-effort step)
6. Step 2 — Move type & home size
7. Step 3 — Date

8. Step 4 — Goods / photos
9. Step 5 — Contact + live summary
10. Thank-you screen
11. Navigation, validation & animation rules
12. Design system — color, type, radii, shadow, spacing tokens
13. Assets manifest — fonts, images, icons
14. External APIs used (both free, no key, called from the browser)
15. Django rebuild plan
16. Exact copy (Norwegian strings), field by field

Priority per your instructions: the main lead form (steps 1–5 + submit) is the thing to build first. Sections 4–11 are the functional spec for that; section 15 is the concrete Django translation plan. Everything else on the marketing site (home page, blog, etc.) is out of scope here.

1. What this page does

`/wizard` is a full-screen, 5-step lead-capture funnel in Norwegian. It collects enough detail about a house move — origin/destination, type of move, home size, date, inventory, and contact info — to hand off to (per the on-screen copy) three moving companies who will each send a quote by email within ~24 hours. It supports being pre-filled via query string: `?fra=<address>` and `?by=oslo|bergen|trondheim|stavanger|tromso` (from `lib/cities.ts`) center the map on a known city before the user has typed anything.

It is a single client-side state machine — one JS object (`WizardData`) mutated as the user moves forward/back through 5 steps, rendered as one continuous "card" UI with a progress bar, not five separate pages/URLs. Step number lives in component state only; refreshing the browser resets to step 1.

2. Critical fact: there is currently no backend

The existing Next.js app never sends this data anywhere. The submit handler (`app/wizard/page.tsx` , function `submit()`) is:

```
const submit = () => {  
  console.log("Wizard submitted:", data);  
  setSubmitted(true);  
};
```

There is no `/api/*` route in the repo, no fetch call, no email integration, no database write. Uploaded photos never leave the browser — they only exist as `URL.createObjectURL` blob previews in memory. **Designing the real submission endpoint (validation, persistence, file storage, and notifying the 3 agencies) is 100% new work for the Django rebuild** — there is no existing behavior to copy for that part, only the front-end contract described in §4 (which fields exist and what they must contain) to satisfy.

3. Page shell & layout

The wizard renders as a `fixed inset-0` overlay (it covers the whole viewport, independent of any normal page scroll) with a centered white "card":

- **Card:** `max-width: 1060px`, `width: calc(100% - 3rem)`, `border-radius: 14px`, background `--surface-soft (#faf9f6)`, shadow `0 20px 60px rgba(0,0,0,.28)`, `0 4px 16px rgba(0,0,0,.10)`, `min-height: 640px` on large screens (≥ 1024 px, Tailwind's `lg:`).
- **Desktop (≥ 1024 px):** card splits into two flex columns: left = form (58% width, generous padding, 2.75rem on largest breakpoint), right = flex-1 panel showing either the live map (step 1), a static step photo (steps 2–4), or the live summary/receipt (step 5). The whole page background is a fixed photo (`boxes-and-plants.jpg`) at 85% opacity of `--bg` over it (i.e. a light wash, not dark). The Kobly wordmark logo floats top-center, absolutely positioned, links to `/`.
- **Mobile (< 1024 px):** the right column is hidden entirely. Instead, the full page background becomes a *step-specific* photo with a dark gradient scrim (`from-black/75 via-black/55 to-black/70`), and it crossfades (400ms) whenever the step's photo changes — implemented by keeping the outgoing image mounted opacity-1 underneath the incoming image (which fades in via a 260ms keyframe), then unmounting the old one after 400ms. The logo sits inline above the card (dark-on-light "brand-ink" tone) instead of overlaid on the photo.

Two intentional exceptions in the mobile background map: step 1 (map) and step 5 (summary) reuse the calm "boxes-and-plants" image instead of a step photo, because those steps don't have a matching desktop lifestyle photo. Step 4's real desktop photo is very dark (avg. brightness 39/255) and reads as almost black behind the scrim, so mobile substitutes step 1's photo there instead. Keep this mapping table when porting:

step 1 → <code>boxes-and-plants.jpg</code>	step 2 → <code>R1-07829-0034.jpg</code>
step 3 → <code>rull3_26.jpg</code>	step 4 → <code>R1-09476-0023-kopi.jpg</code> (NOT <code>R1-09476-0028.jpg</code>)
step 5 → <code>boxes-and-plants.jpg</code>	

Desktop step-image map (right column, steps 2–4 only): 1→`R1-09476-0023-kopi.jpg`, 2→`R1-07829-0034.jpg`, 3→`rull3_26.jpg`, 4→`R1-09476-0028.jpg`.

Inside the left column, top to bottom: a 5-segment progress bar (thin rounded bars, filled = `bg-ink`, unfilled = `bg-ink/10`, `i < step` is filled i.e. completed steps only, not the current one), a "Steg X av 5" label, the animated step content, and a bottom nav bar pinned via `absolute bottom-6 left-6 right-6` with a "Tilbake" text link (hidden on step 1) on the left and the primary "Neste"/"Send forespørsel" button on the right.

4. State model — the full data shape

```
type WizardData = {  
  flytteType: "privat" | "bedrift" | "internasjonal" | "";  
  fra: string; // free-text address, from-side  
  fraCoord: { lat: number; lon: number } | null;  
  til: string; // free-text address, to-side  
  tilCoord: { lat: number; lon: number } | null;  
  boligtype: "leilighet" | "rekkehus" | "enebolig" | "annet" | "";  
  flyttedato: string; // yyyy-mm-dd from <input type="date">, "" if none  
  fleksibel: boolean; // "I'm flexible on date" toggle  
  beskrivelse: string; // free-text inventory description  
  bilder: File[]; // uploaded photos, client-only today  
  navn: string;  
  telefon: string;  
  epost: string;  
};
```

This whole object is what the Django form/model needs to accept on final submit (plus the photo files as multipart uploads). It is built incrementally: every field starts empty/false/null, and each step's inputs write directly into this one object — there is no per-step sub-form or intermediate save step in the current app.

5. Step 1 — "Hvor skal du flytte?" (address + map)

Header: **Hvor skal du flytte?** / subtitle: *Skriv inn fra-adresse og til-adresse. Valid when: both `fra` and `til` are >2 characters (coordinates are NOT required to advance).*

5.1 Address fields with autocomplete

Two labeled text inputs, "Fra adresse" and "Til adresse", placeholders `F.eks. Kongens gate 1, 0153 Oslo` / `F.eks. Storgata 14, 0184 Oslo`. Behavior:

- On every keystroke, once the field has focus and ≥ 2 characters, a request fires after a **200ms debounce** to `GET https://ws.geonorge.no/adresser/v1/sok?sok={query}&treffPerSide=8&side=0` (Kartverket's official Norwegian address registry — free, public, no API key). In-flight requests are aborted via `AbortController` if the user keeps typing.
- Up to 8 results render in a dropdown below the field: each row shows `adressesstekst` bold and `postnummer` `poststed` small/muted underneath.
- Clicking a suggestion sets the field text to `"{adressesstekst}, {postnummer} {poststed}"` and stores its `representasjonspunkt` lat/lon as the coordinate. A "just selected" flag suppresses re-firing the search on the resulting value change.
- The "Fra" field only (not "Til") additionally shows a pinned first row "Bruk min plassering" (use my location) with a map-pin icon, using `navigator.geolocation.getCurrentPosition`, then reverse-geocoding the coordinate (see §14) to fill the text with the label `"Min posisjon"` as fallback.
- Typing manually (not selecting a suggestion) clears any previously attached coordinate (`onChange(value, null)`).
- Dropdown closes on blur (120ms delayed, to allow the click on a suggestion to register first).

5.2 Map (desktop: inline panel • mobile: fullscreen picker)

Built on **Leaflet** with **CARTO Voyager** raster tiles (no key needed):

`https://{s}.basemaps.cartocdn.com/rastertiles/voyager/{z}/{x}/{y}{r}.png`, `zoomControl` and `attributionControl` both disabled (custom UI is built instead — see below). Initial center: URL-provided city from `lib/cities.ts` if present, else Norway-wide view `[60.5, 10.0]` zoom 5.

Desktop (`MapPanel`, right column of step 1):

- Custom zoom in/out buttons top-right; a "use my location" button bottom-left (pulses while locating); two pill toggle buttons bottom-center, "Plasser fra" / "Plasser til" — clicking arms "placing mode" (cursor becomes crosshair over the map, via a `.placing-mode` CSS class), and the next map click drops/moves that pin, reverse-geocodes it, and disarms placing mode.
- Pins are draggable Leaflet markers with custom circular divlcons: "from" pin is dark (`#221814`), "to" pin is olive-green (`#3D5507`), both a location-pin glyph in a 36px white-bordered circle. Dragging live-updates the connecting line; drag-end reverse-geocodes the new position.
- A 250m translucent circle is drawn around the "from" pin (fill `#221814` @ 8% opacity).
- When both pins exist: a dashed line (`#221814`, 6/6 dash) connects them, with a small white circular "Kobly ring" icon (the brand mark) at the exact midpoint; the map auto `fitBounds`s to both pins with 40px padding, max zoom 13.
- A small floating chip top-left shows "Ca. {fromShort} → {toShort}" once either address has text (only the first comma-segment of each address is shown, e.g. street only, not city/postcode).
- Bottom-right: an info "i" button revealing a tooltip "© Leaflet · © CARTO · © OpenStreetMap" on hover — attribution is required by CARTO's free tile terms even with `attributionControl` visually off.

Mobile (`MapPickerOverlay`, opened via a "Plasser i kart" pill under each address field):

- Fullscreen modal (reuses a generic `FullscreenOverlay` : locks body scroll, closes on Escape, `role="dialog"`). A fixed crosshair pin sits dead-center of the screen; the user pans the *map* underneath it instead of dragging a marker.
- On open and on every `moveend` , the map's current center is reverse-geocoded and shown in a bottom sheet ("Valgt plassering" / the address, or "Finner adresse ..." while loading, dimmed while re-fetching).
- Top bar: title + instruction text ("Flytt kartet så nålen står på adressen din") + close (X) button. A "use my location" floating button sits above the bottom sheet.
- Confirm button: "Bruk denne plasseringen" (full-width, brand-filled, pill) — commits the current map center + resolved address back to the wizard step and closes the overlay.
- The "Plasser i kart" trigger button itself changes state once a pin exists: label switches to "Plasseringen valgt · endre i kart" with a checkmark and a tinted lime background instead of the neutral gray default.

6. Step 2 — "Hva slags flytting er det?" (type & size)

No subtitle. Two labeled groups, both required to advance:

Type (3-column pill grid)

- Privat
- Bedrift
- Internasjonal

Single-select. Unselected pill = light tan fill

#F3EEE3 , muted text; selected = solid brand-black fill, cream text, brand border.

Størrelse (2-column pill grid)

- Leilighet
- Rekkehus
- Enebolig
- Annet

Same single-select pill visual language, independent of the Type selection above.

7. Step 3 — "Når skal du flytte?" (date)

Subtitle: *Velg en dato eller la oss vite om du er fleksibel.* Two mutually-exclusive controls:

- A native `<input type="date">` labeled "Ønsket flyttedato" — selecting a date automatically clears the "flexible" toggle.
- A dashed pill toggle button "Jeg er fleksibel på datoen" with a checkbox-style circle — turning it ON clears any chosen date and visually disables/dims (50% opacity) the date input; turning it back OFF re-enables the date input.

Valid to advance: a date is chosen **or** flexible is true (never both required, never both empty).

8. Step 4 — "Hva skal du flytte?" (goods / photos)

Subtitle: *Legg til bilder eller en beskrivelse av tingene dine.* Always valid — this step is optional and never blocks progression. Two inputs:

- **Beskrivelse**: a 4-row textarea, placeholder "*F.eks. 3-seters sofa, stort spisebord, 2 senger, 10 esker...*", no character limit enforced client-side.
- **Bilder** (subtext: "Legg ved bilder av tingene dine for et mer presist tilbud."): a 4-column grid of square thumbnails. Each uploaded file becomes a thumbnail (object-fit cover, rounded 14px) with a small circular "x" remove button top-right. The last grid cell is always a dashed-border "Last opp" (upload) tile — a label wrapping a hidden `<input type="file" multiple accept="image/*">` — so users can keep adding more. New files are appended to the existing array, never replace it. Previews are `URL.createObjectURL` blobs, explicitly revoked on unmount/change to avoid leaking memory — there is no upload/compression step; the raw `File` objects sit in memory until final submit.

9. Step 5 — "La oss ta kontakt" (contact) + live summary

Subtitle: *Vi kobler deg med tre byråer. Du hører fra dem innen 24 timer.* Three fields, 2-column grid on ≥640px (email spans both columns):

Field	Placeholder	Input type	Validation to submit
Navn	Ola Nordmann	text	<code>trim().length > 1</code>
Telefon	+47 000 00 000	tel (inputMode)	<code>/^[\\d\\s+]{8,}\$/</code> — digits, spaces, "+" only, min 8 chars
E-post	ola@eksempel.no	email	<code>/\\S+@\\S+\\.\\S+\\/</code> — permissive, not RFC-strict

The button on this step reads **"Send forespørsel"** instead of "Neste" (it's the last step; clicking it, once valid, calls `submit()`).

Right column on desktop (step 5 only) is a live receipt, not a static image — it re-renders from the same `data` object in real time as the user types on the left:

- Heading "Kvittering" / "Din forespørsel", note "Vi sender denne til tre kvalitetssjekkede byråer i ditt område."
- Conditionally-rendered rows (only shown if the underlying data exists): Fra, Til, "Flytting" (type · size joined with " · "), "Når" (formatted Norwegian long date e.g. "3. september 2026", or "Fleksibel dato"), "Innhold" (the description, trimmed), and a photo thumbnail grid if any images were added.

10. Thank-you screen

Replaces the entire wizard (not a step — a separate top-level render branch once `submitted === true`). Centered card over the same darkened `boxes-and-plants.jpg` background (65% black overlay, no scrim gradient this time — flat). Contents:

- A small "routing" illustration: Kobly's ring logo (in a solid brand-filled circle) → three dots → three lime-green rounded squares each containing a checkmark/box icon, popping in staggered (120ms, 230ms, 340ms delays) with a bouncy `cubic-bezier(0.34,1.56,0.64,1)` scale-in.
- Heading: **Forespørselen er sendt!**
- Body: *"Vi har nå sendt forespørselen din til tre kvalitetssjekkede byråer i ditt område. Du mottar tilbud på e-post innen kort tid."*
- Full-width pill button, brand-filled: **Tilbake til forsiden** → links to `/`.

11. Navigation, validation & animation rules

- "Neste"/"Tilbake" are the only way to change steps — no clickable progress-bar segments, no step numbers as links, no swipe gesture. "Tilbake" is a plain text link, present on steps 2–5 only.
- The primary button is rendered but `disabled` (visually: gray, `cursor-not-allowed`, no hover) whenever the current step's validity check fails — it never navigates on invalid input; there is no error-message/shake state, just the disabled affordance.
- Direction of travel (forward vs. back) sets which slide-in keyframe plays for the new step's content: forward → content slides in from the right (`translateX(24px)→0`), back → from the left (`-24px→0`), both 260ms ease, combined with an opacity fade. The right-column art/map/summary panel independently does a plain 260ms fade (no slide) keyed on the step number.
- State is never persisted (no localStorage/sessionStorage/URL step param) — a page refresh loses all progress and returns to step 1. Worth deciding explicitly whether the Django version should add persistence (e.g. session-backed) since server-rendered multi-page steps make a refresh-losing-state experience feel worse than in a SPA.

12. Design system

12.1 Color tokens

--bg

#f4f1ea

--surface

#ffffff

--surface-soft

#faf9f6

--ink

#1a1a1a

--ink-muted

#5c5c5c

--line

#e6e1d6

--brand (primary!)

#221814

--brand-ink

#faf9f6

--accent-lime

#d6e8a8

--accent-amber

#e8a87c

--move (unused on this page)

#2f5fa8

Note the brand color is a near-black brown (`#221814`), not blue/green — it's used as the "primary" fill for CTAs, selected pills, and the "from" map pin. The olive green `#3D5507` (hardcoded, not a CSS var) is used specifically for the "to" map pin and the checkmark icon color in the thank-you illustration. Most text hierarchy is done via opacity on `--ink` (e.g. "ink/40", "ink/45", "ink/50") rather than distinct gray tokens — reproduce as `rgba(26, 26, 26, 0.40)` etc., or `color-mix` in SCSS.

12.2 Typography

Role	Family	Source	Used for
--font-sans	Moderat	Local <code>.otf</code> files, weights 400 & 600 only (<code>app/fonts/Moderat-Regular.otf</code> , <code>-Semibold.otf</code>)	Body text, labels, buttons — the default
--font-serif / --font-heading	Crimson Pro	Google Font	All step <code><h1></code> headings, e.g. "Hvor skal du flytte?" — 3xl→4.75xl responsive, weight 500, tight tracking
--font-display	Syne	Google Font	Loaded but not used anywhere on this specific page

A deliberate detail: the CSS explicitly avoids enabling the `ss01` OpenType feature (only `cv11` is on) because it swaps `? / & / %` glyphs for alternates that don't exist in the Semibold weight, causing punctuation to silently fall back to Regular and look thin next to bold text. If you license/self-host Moderat for the Django static files, keep this in mind when setting `font-feature-settings` in SCSS.

12.3 Shape, spacing, elevation

Token	Value	Where
Card radius	14px	Wizard card, inputs, pills, block-cards, thumbnails

Full pill radius	9999px	Toggle buttons, CTA buttons, map "placer" pills
Input border	1.5px solid rgba(26,26,26,.10), fill #F7F5F1, focus border → brand	All text inputs/textarea/date
Card shadow (wizard)	0 20px 60px rgba(0,0,0,.28), 0 4px 16px rgba(0,0,0,.10)	Main wizard card
Small UI shadow	0 1px 4px rgba(0,0,0,.12-.15)	Map buttons, address chip
Step transition	260ms ease, translateX 24px + opacity	Left column step content
Mobile bg crossfade	400ms	Background photo swap between steps

13. Assets manifest

13.1 Fonts to carry over

- `Moderat-Regular.otf` , `Moderat-Semibold.otf` — proprietary, currently in `app/fonts/` ; must be re-licensed/copied for Django's static files (self-hosted `@font-face` , not Google Fonts).
- Crimson Pro — free on Google Fonts, can self-host or link.

13.2 Images used specifically by /wizard

<code>public/images/boxes-and-plants.jpg</code>	– page background (desktop, all steps) + mobile step 1 & 5 + thank-you bg
<code>public/images/R1-09476-0023-kopi.jpg</code>	– desktop step 1 art slot / mobile step 4 bg
<code>public/images/R1-07829-0034.jpg</code>	– desktop step 2 art slot / mobile step 2 bg
<code>public/images/rull3_26.jpg</code>	– desktop step 3 art slot / mobile step 3 bg
<code>public/images/R1-09476-0028.jpg</code>	– desktop step 4 art slot (desktop only, too dark for mobile scrim)

13.3 Icons (lucide-react → replace with static SVG/icon font)

<code>ArrowRight, Check, Info, MapPin, Send, Upload, X, ZoomIn, ZoomOut</code>	<code>(page.tsx)</code>
<code>Crosshair, LocateFixed, Check, X</code>	<code>(MapPickerOverlay.tsx)</code>

All are simple stroke-based line icons (24×24 viewBox, `currentColor`). Since Lucide ships raw SVG per icon, the least-effort port is to inline the exact SVG paths as static markup/SCSS-colored `<svg>` in the Django templates — no icon library/JS dependency required.

14. External APIs used (client-side, free, no key required)

Purpose	Endpoint	Notes
Address autocomplete	GET <code>https://ws.geonorge.no/adresser/v1/sok?sok={q}&treffPerSide=8&side=0</code>	Kartverket (Norwegian Mapping Authority) official address DB. Returns <code>adresser[]</code> with <code>adresstekst</code> , <code>postnummer</code> , <code>poststed</code> , <code>representasjonspunkt</code> , <code>{lat,lon}</code> .
Reverse geocode	GET <code>https://ws.geonorge.no/adresser/v1/punktsok?radius=200&lat={lat}&lon={lon}&treffPerSide=1&side=0</code>	Same service, nearest-address-to-point. Falls back to a friendly string ("Pin plassert i kart" / "Min posisjon") on error or empty result — never blocks the UI.
Map tiles	<code>https://{s}.basemaps.cartocdn.com/rastertiles/voyager/{z}/{x}/{y}{r}.png</code>	CARTO Voyager raster tiles, free tier, attribution required (see §5.2).

These are all called directly from the browser (no server proxy today) and are CORS-open, so the vanilla-JS port can call them exactly the same way client-side — Django doesn't need to proxy them unless you want to hide/rate-limit the calls or add server-side caching.

15. Django rebuild plan

Concrete translation of the above into Django + server-rendered HTML/SCSS/vanilla JS, no React.

15.1 Data model

```
class MoveLead(models.Model):
    FLYTTE_TYPE_CHOICES = [("privat", "Privat"), ("bedrift", "Bedrift"), ("internasjonal", "Internasjonal")]
    BOLIG_TYPE_CHOICES = [("leilighet", "Leilighet"), ("rekkehus", "Rekkehus"),
                          ("enebolig", "Enebolig"), ("annet", "Annet")]

    flytte_type = models.CharField(max_length=20, choices=FLYTTE_TYPE_CHOICES)
    fra_adresse = models.CharField(max_length=255)
    fra_lat = models.FloatField(null=True, blank=True)
    fra_lon = models.FloatField(null=True, blank=True)
    til_adresse = models.CharField(max_length=255)
    til_lat = models.FloatField(null=True, blank=True)
    til_lon = models.FloatField(null=True, blank=True)
    bolig_type = models.CharField(max_length=20, choices=BOLIG_TYPE_CHOICES)
    flyttedato = models.DateField(null=True, blank=True)
    fleksibel = models.BooleanField(default=False)
    beskrivelse = models.TextField(blank=True)
    navn = models.CharField(max_length=120)
    telefon = models.CharField(max_length=30)
    epost = models.EmailField()
    created_at = models.DateTimeField(auto_now_add=True)

class MoveLeadImage(models.Model):
    lead = models.ForeignKey(MoveLead, related_name="images", on_delete=models.CASCADE)
    image = models.ImageField(upload_to="leads/%Y/%m/")
```

15.2 Multi-step pattern without React

Two viable approaches — pick one:

- **A. Single template, JS-driven steps (closest match to current UX).** Render all 5 step `<section>`s in one page, hidden/shown via a small vanilla-JS controller (toggle a `data-step` attribute, CSS handles the slide/fade with the same 260ms keyframes). Keep one JS object mirroring `WizardData`, mirrored into hidden form inputs so the final POST carries everything at once. This preserves the animated, no-reload, single-URL feel exactly.
- **B. Real server-rendered multi-page wizard** (e.g. `django-formtools` `SessionWizardView`), one Django view/URL per step, session-backed state, full page (or HTMX partial) navigation between steps. More idiomatic Django, gives you free "resume after refresh" (the current app can't do this), but you'd rebuild the transition animation via CSS view-transitions or accept plain page loads.

Given the priority is fidelity to this exact form/funnel, **Option A** is the closer match; Option B is the more "correct" Django-native approach if you're open to a slightly different feel. Either way, server-side validation in `forms.py` should re-implement the same per-step rules from §4–9 verbatim (min lengths, the phone regex, the email regex, the date-or-flexible XOR-ish rule) — the client-side JS validation is only for UX (disabling the button); it must not be trusted as the source of truth.

15.3 Address autocomplete + map in vanilla JS

- Same Geonorge endpoints from §14, called with plain `fetch()` + a hand-rolled 200ms debounce (`setTimeout` / `clearTimeout`) and `AbortController` — no framework needed.

- Leaflet itself is framework-agnostic (it's what the current app uses under React, just imperatively via refs) — the exact same Leaflet init/marker/circle/polyline code in §5.2 ports near-verbatim to plain `<script>` JS; you're really just removing the React lifecycle wrapper around it, not rewriting map logic.
- Photo upload: switch from in-memory `File[]` + blob preview to a real `<input type="file" multiple>` inside a standard `multipart/form-data` form; keep the same "grid of thumbnails with an X to remove + trailing upload tile" UI, implemented with the `DataTransfer` API to let users remove a specific file from the input's `FileList` before submit, or simply track "removed" filenames and filter server-side if simpler.

15.4 SCSS token mapping

Port `app/globals.css`'s `:root` variables directly as SCSS variables or CSS custom properties (no need to go through Tailwind's `@theme` indirection) — see the full table in §12.1–12.3. Recreate the four keyframes (`wizard-slide-right`, `wizard-slide-left`, `wizard-fade-in`, `success-pop`) verbatim in plain CSS; they're small and self-contained.

15.5 Submission endpoint (new — doesn't exist today)

This is genuinely new design work, not a port. Minimum viable version:

1. `POST` handler validates the full payload server-side against the rules in §4–9.
2. Persist a `MoveLead` + its `MoveLeadImage`s.
3. Notify "three agencies" — decide now whether that's a real routing/matching system (e.g. by city/region) or, for a first version, a single internal notification email/Slack webhook per new lead, with agency matching added later. The current front-end already promises "3 byråer... innen 24 timer" in copy, so whatever you ship needs to either fulfill that or the copy should be softened.
4. Return success → server renders (or JS swaps in) the same Thank-You screen from §10.

16. Exact copy (Norwegian strings)

For pixel/word-perfect parity, verbatim strings by step:

```
Progress label:      "Steg {n} av 5"

STEP 1
  H1:                "Hvor skal du flytte?"
  Subtitle:          "Skriv inn fra-adresse og til-adresse"
  Field 1 label:     "Fra adresse"           placeholder: "F.eks. Kongens gate 1, 0153 Oslo"
  Field 2 label:     "Til adresse"          placeholder: "F.eks. Storgata 14, 0184 Oslo"
  Autocomplete row:  "Bruk min plassering"
  Map picker (mobile, empty): "Plasser i kart"
  Map picker (mobile, set):   "Plasseringen valgt · endre i kart"
  Map overlay title (fra):    "Hvor flytter du fra?"
  Map overlay title (til):    "Hvor flytter du til?"
  Map overlay hint:         "Flytt kartet så nålen står på adressen din"
  Map overlay confirm:       "Bruk denne plasseringen"
  Map overlay label:        "Valgt plassering" / loading: "Finner adresse ..."
  Map desktop pills:        "Plasser fra" / "Plasser til" / (armed) "Klikk på kartet"
  Map chip prefix:         "Ca."
  Map attribution:          "© Leaflet · © CARTO · © OpenStreetMap"

STEP 2
  H1:                "Hva slags flytting er det?"
  Group 1 label:      "Type"                options: Privat / Bedrift / Internasjonal
  Group 2 label:      "Størrelse" options: Leilighet / Rekkehus / Enebolig / Annet

STEP 3
  H1:                "Når skal du flytte?"
  Subtitle:          "Velg en dato eller la oss vite om du er fleksibel"
  Field label:        "Ønsket flyttedato"
  Toggle:             "Jeg er fleksibel på datoen"

STEP 4
  H1:                "Hva skal du flytte?"
  Subtitle:          "Legg til bilder eller en beskrivelse av tingene dine."
  Field label:        "Beskrivelse"          placeholder: "F.eks. 3-seters sofa, stort spisebord, 2 senger,
10 esker..."
  Photos label:       "Bilder"
  Photos hint:        "Legg ved bilder av tingene dine for et mer presist tilbud."
  Upload tile:        "Last opp"

STEP 5
  H1:                "La oss ta kontakt"
  Subtitle:          "Vi kobler deg med tre byråer. Du hører fra dem innen 24 timer."
  Fields:            "Navn" (Ola Nordmann) · "Telefon" (+47 000 00 000) · "E-post" (ola@eksempel.no)
  Submit button:     "Send forespørsel"

RIGHT-COLUMN SUMMARY (desktop, step 5)
  Eyebrow:           "Kvittering"
  Heading:           "Din forespørsel"
  Note:              "Vi sender denne til tre kvalitetssjekkede byråer i ditt område."
  Row labels:        "Fra" / "Til" / "Flytting" / "Når" / "Innhold" / "Bilder"
  "Når" flexible value: "Fleksibel dato"

NAV
  Back:              "Tilbake"
  Next:              "Neste"
```

THANK-YOU SCREEN

Heading: "Forespørselen er sendt!"
Body: "Vi har nå sendt forespørselen din til tre kvalitetssjekkede byråer i ditt område. Du mottar tilbud på e-post innen kort tid."
Button: "Tilbake til forsiden"

Generated from a direct read of the Kobly Next.js source (app/wizard/page.tsx and related components) — not from the rendered screenshot alone.